

TP6 - Sistemas Operativos

1. ¿Qué parámetro del sistema se ve afectado directamente por la administración de memoria?

Lo que se ve afectado es la CPU, ya que con la aparición del concepto de multiprogramación, buscamos mejorar su utilización y la velocidad de respuestas para los usuarios. Para que se pueda lograr este desempeño se deben mantener varios procesos en memoria, para lo cual necesitamos una buena administración de la memoria. El administrador de memoria a su vez, como busca hacerlo de la manera más eficiente tiene que llevar registros de los cuáles partes de la memoria están en uso, asignar memoria a los procesos cuando la necesiten y desasignar cuando terminan.

Info extra: La memoria principal y los registros integrados en cada núcleo de procesamiento son el único almacenamiento de propósito general al que la CPU puede acceder directamente.

2. ¿Cuál es la diferencia entre una dirección física y una dirección virtual (dirección lógica)?

Román Glosario Silberschatz & Silberschatz 9.1.3

La diferencia entre una dirección física y lógica es que la primera corresponde a una dirección real de la memoria, en cambio la dirección lógica es una dirección generada por el CPU que corresponde a la memoria virtual y esta debe ser previamente traducida a una dirección física antes de ser usada.

Los programas de usuario no ven las direcciones físicas reales, sino que trabajan y generan direcciones lógicas. Sin embargo, estas direcciones lógicas deben ser mapeadas a direcciones físicas antes de ser utilizadas.

Name two differences between logical and physical addresses.

Answer:

A logical address does not refer to an actual physical address; rather, it refers to an abstract address in an abstract address space. A physical address refers to an actual physical address in memory. A logical address is generated by the CPU and is translated into a physical address by the memory management unit(MMU). Therefore, physical addresses are generated by the MMU.

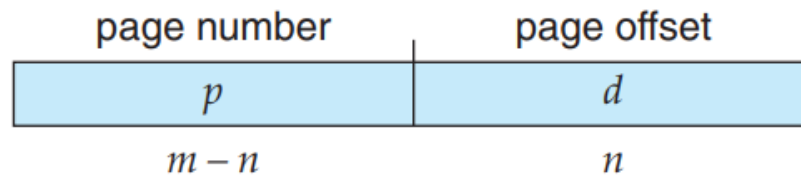
3. Internamente ¿qué tipo de direcciones maneja la CPU?

Direcciones lógicas/virtuales.

4. ¿Por qué el número de páginas y el tamaño de las páginas es de potencia de 2?

Roman Silberschatz 9.3.1

El número de páginas y el tamaño de las páginas es de potencia de 2 porque facilita la traducción de una dirección lógica en un número de página y el offset. Si el tamaño del espacio de direccionamiento lógico es 2^m , y el tamaño de la página es 2^n bytes, entonces para el número de página se destinan $m-n$ bits y para el offset n bits



Como cada posición de un bit representa una potencia de 2, dividir una dirección entre bits resulta en un tamaño de página que es potencia de 2.

Why are page sizes always powers of 2?

Answer:

Recall that paging is implemented by breaking up an address into a page and offset number. It is most efficient to break the address into X page bits and Y offset bits, rather than perform arithmetic on the address to calculate the page number and offset. Because each bit position represents a power of 2, splitting an address between bits results in a page size that is a power of 2.

5. *Explique la diferencia entre fragmentación interna y fragmentación externa.*

Silberschatz 9.2.3

La fragmentación interna hace referencia a cierto espacio de memoria desperdiciado por un programa, ya que por ejemplo, el tamaño de bloque es más grande que el programa en sí, pero no se le puede asignar menos espacio. Entonces, el programa tiene asignado cierto espacio que nunca va a usar. En otras palabras, es memoria que no se va a usar dentro de una partición. Ocurre normalmente cuando hay paginado, o asignación contigua de posicionamiento fijo.

Por otro lado, la fragmentación externa ocurre cuando, al irse asignando y removiendo bloques de memoria, quedan pequeños espacios entre bloques que, individualmente no pueden albergar a ningún programa, pero en conjunto si podrían. Esto ocurre normalmente cuando hay algún tipo de segmentación, o asignación contigua de posicionamiento dinámico.

Fragmentación interna: la memoria ocupada puede ser ligeramente más grande que la demandada; esta diferencia de tamaño es memoria interna de la partición, pero no es usada.

Fragmentación externa: existe espacio de memoria para satisfacer una demanda, pero no es contiguo.

6. ¿Qué ventajas tiene la utilización de páginas multinivel?

Silberschatz 9.4.1

Muchos sistemas de computación modernos soportan un gran espacio de direcciones lógicas, por lo cual la tabla de páginas se vuelve excesivamente grande. No es deseable alojar la tabla de páginas en forma contigua en memoria principal. Utilizar páginas multinivel permite solucionar este problema dividiendo la tabla de páginas en piezas más pequeñas, de forma tal que no se necesita tener toda la tabla de página cargada en memoria, ahorrando espacio en memoria.

Particionando la tabla de páginas de esta manera, el sistema operativo puede dejar particiones no usadas hasta que un proceso las necesite. Frecuentemente secciones enteras del espacio de direcciones virtuales no son usadas. Las tablas de páginas multinivel no tienen entradas por esos espacios, disminuyendo en gran medida la cantidad de memoria necesaria para almacenar estructuras de datos de memoria virtual.

Se puede tener tablas de páginas de varios niveles, por ejemplo, tener una tabla de directorios de páginas cuyas entradas apuntan a otras tablas de páginas, y ahí sí apuntan a una tabla de páginas que tiene los frames. Entonces, se tienen $2 \times 2 \times 2 = 2$ bits direccionables.

7. Analice los modelos de memoria: paginada, segmentada y segmentación con paginación. Discuta las ventajas y desventajas de cada uno de ellos.

Consideration	Paging	Segmentation
Need the programmer be aware that this technique is being used?	No	Yes
How many linear address spaces are there?	1	Many
Can the total address space exceed the size of physical memory?	Yes	Yes
Can procedures and data be distinguished and separately protected?	No	Yes
Can tables whose size fluctuates be accommodated easily?	No	Yes
Is sharing of procedures between users facilitated?	No	Yes
Why was this technique invented?	To get a large linear address space without having to buy more physical memory	To allow programs and data to be broken up into logically independent address spaces and to aid sharing and protection

Sigue otra tabla abajo...

	Paginación	Segmentación	Segmentación con paginación
Ventajas	• Permite que el espacio de direcciones físicas de un proceso no sea contiguo. • Evita fragmentación externa y la necesidad de compactar. • Resuelve el problema de tener el almacenamiento externo dividido en trozos de distintos tamaños. • Clara separación entre la memoria vista por el programador y la memoria física real. • Facilita la vinculación dinámica de librerías cargando las páginas que se mapean al mismo espacio físico.	• Soporta la vista de la memoria de un programador: colección de segmentos de tamaño variable, sin orden necesario entre los segmentos. • Permite que el espacio de direcciones físicas de un proceso sea continuo. • No genera fragmentación interna porque otorga el espacio justo a cada objeto. • Simplifica el manejo de estructuras de datos que aumentan o reducen su tamaño. • Para compartir memoria solo se debe poseer el mismo número de segmentos. • Los distintos segmentos pueden tener diferentes tipos de protección.	• Mejor manejo de segmentos. Soporta la vista de usuario a la vez que la página
Desventajas	• Si el tamaño de los procesos es independiente del tamaño de página, se espera fragmentación interna en promedio de media página por proceso. • Produce un acceso extra a memoria para acceder a la tabla y luego a la dirección real (TLB lo atenúa).	• Sufre de fragmentación externa. • El programador debe estar atento a la limitación máxima de tamaño de segmento. • No hay una relación simple entre las direcciones físicas y lógicas. • Dado que los segmentos varían en longitud, la alocaión de memoria es un problema de alocaión dinámica de almacenamiento. Para este problema se acude a la utilización de la política de primer lugar y mejor lugar. Claramente esto produce problemas de fragmentación externa.	• El tiempo de acceso a memoria es mayor porque hay que acceder a la tabla de segmentos y luego a la de páginas.

8. ¿Cómo se protegen los esquemas de asignación del ejercicio 7?

- Paginación: se proporcionan bits de protecciones asociados a cada frame, los cuales se encuentran normalmente en la tabla de páginas. Además podemos poseer un hardware para proveer solo lectura (read-only), solo escritura (read-write) o solo ejecución (execute-only), o tener un bit de protección por cada tipo de acceso.

Otro bit adicional se adjunta a cada entrada de la tabla de páginas, el bit de válido/inválido. Cuando este es seteado en válido, la página asociada se encuentra en el espacio de direcciones lógicas de un proceso y es una página legal. Cuando es inválido se trata de una dirección ilegal donde no sucede lo anterior.

También se utiliza un registro PTLR (registro de longitud de la tabla de páginas) como mecanismo de protección. Este indica el tamaño de la tabla de páginas y se compara con cada dirección lógica para verificar que esa dirección se encuentra dentro del rango de direcciones válidas definidas para el proceso.

- Segmentación: este se protege teniendo en cada entrada de la tabla de segmentos un bit de validación el cuál nos indica con 0 si el segmento es ilegal. Además se puede otorgar los privilegios read/write/execute. Es decir, los bits de protección se los asocia a los segmentos, y el código compartido ocurre a nivel de segmento. Hay que tener cuidado ya que cómo los segmentos varían en longitud, la aloación de memoria es un problema de aloación dinámica de almacenaje.

9. La mayoría de los sistemas permiten a los programas asignar más memoria a su espacio de direcciones durante la ejecución. Como ejemplo de ese tipo de asignación de memoria tenemos los datos asignados en los segmentos de los programas dedicados a un cúmulo de memoria. ¿Qué se necesitaría para soportar la asignación dinámica de memoria en los siguientes esquemas?

a. asignación contigua de memoria: podría requerir la reubicación de todo el programa, ya que no hay suficiente espacio para que el programa aumente su espacio de memoria asignado. Habría que diferenciar entre ac dinámica y de posicionamiento fijo?

b. Segmentación: al igual que antes, se podría requerir reubicar todo el programa ya que no hay suficiente espacio para que el segmento aumente su espacio de memoria asignado.

c. Paginación: la asignación incremental de nuevas páginas es posible en este esquema sin requerir la reubicación del espacio de direcciones del programa.

10. Analice qué ventajas y desventajas se obtiene de la utilización de páginas grandes y de páginas pequeñas.

Con páginas grandes, se tiene la ventaja de una tabla de páginas más pequeña, ya que van a existir menor cantidad de páginas. No obstante, la cantidad de memoria perdida en

fragmentación interna podría ser mayor. Además, una ventaja es que normalmente, cuando se accede a un byte de una página se accede a su entorno, por lo que al tener la página más grande, se va a acceder menos a disco. No obstante, se podrán tener menos cantidad de páginas en memoria.

Contrariamente, con menor tamaño de página se tiene una tabla de páginas más grandes, pero la cantidad de memoria perdida en fragmentación interna podría ser menor. Además, se deberán traer más paginas, pero habrá lugar para mayor cantidad en memoria. Se podría considerar también lo de localidad espacial y localidad temporal. **Revisar Silbershatz 10.9.2 Page Size**

11. El diagrama de la figura 1 muestra un ejemplo del estado de memoria utilizando administración de alocaación contigua dinámica, después de varias operaciones de reemplazo y swapping-out ocurridas en el sistemas. Las direcciones van de izquierda a derecha; las áreas grises representan a procesos que están cargados en memoria; y las áreas blancas indican bloques de memoria libre. El último proceso cargado en memoria requirió 2 Mbytes y está marcado con una X. Solamente una operación de swap-out ocurrió después de la última carga.

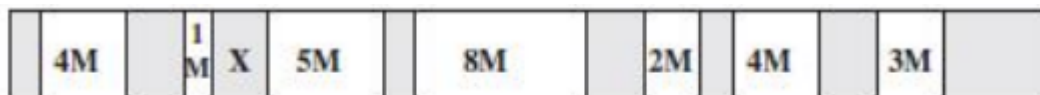


Figura 1: Configuración de memoria

a. ¿Cuál fue el tamaño máximo del proceso intercambiado?

Si el proceso intercambiado hace referencia al proceso que fue swapped-out, entonces el máximo tamaño es 8 (ya que hay como mucho 8 M libres).

Encontré lo mismo. El tamaño máximo del proceso intercambiado se puede determinar observando el bloque de memoria libre más grande después de que se colocó el último proceso. Y la explicación era que la operación de intercambio habría creado un bloque de memoria libre del mismo tamaño que el proceso intercambiado.

b. ¿Cuál era el tamaño del bloque libre justo antes de que X lo particionara?

El tamaño del bloque libre era de $1M + 5M + 2M = 8M$.

c. Una nueva solicitud de asignación de 3 Mbytes es requerida. Indicar los bloques de memoria donde se creará una partición para el nuevo proceso considerando los cuatro algoritmos de aplicación dinámica:

Mejor ajuste: Usa el último hueco de 3MB (el más pequeño que puede tener al proceso).

Primer ajuste: Usa el hueco libre de 4 MB (el primero).

Siguiente ajuste: Lo ubica en el de 5 MB (el primer hueco libre que se encuentra después de ubicar al último proceso).

Peor ajuste: Lo ubica en el de 8 MB.

12. En un sistema con paginación, un proceso no puede acceder a una zona de memoria que no sea de su propiedad. ¿Por qué? ¿Cómo podría el sistema operativo permitir el acceso a otras zonas de memoria? ¿Por qué debería o porque no debería?

9.3.3 Silberschatz

Un proceso no debería acceder a una zona de memoria que no sea de su propiedad, debido a que podría modificar datos que el proceso que es propietario de esa sección de memoria está usando, lo que ocasionaría el mal funcionamiento del mismo. Si esto se extiende, podría ocasionar que ningún proceso funcione correctamente, o que se acceda y se lea información privilegiada, lo cual pone en riesgo el funcionamiento del sistema entero.

Para hacer que varios procesos puedan acceder a una zona de memoria común, el SO podría hacer uso del concepto de páginas reentrantes, las cuales son páginas a las que varios procesos pueden acceder al mismo tiempo. Tienen la ventaja de permitir que varios procesos las lean, pero para asegurar eso mismo, no pueden ser modificadas (se usan por ejemplo, para librerías comunes). Al hacer uso de las mismas, se podría estar ahorrando espacio. Si 10 procesos usaran una librería que pesa 2MB, que cada uno tenga su propia copia sería un total de 20MB. En cambio, si todos la comparten, el espacio usado es de tan solo 2MB, mejorando la utilización de los recursos ahorrándonos 18MB.

No obstante, esto trae sus complicaciones. Supongamos que dos programas A y B están compartiendo una página (de A), y A es reemplazado. Esto ocasionaría que numerosas veces B tenga page faults (mientras A siga siendo reemplazado), lo que gasta mucho tiempo. Además, si A es finalizado, hay que tener cuidado que no se borren las páginas que B está usando. En el caso de las librerías compartidas, hay que tener cuidado con las referencias, ya que podría pasar que el código sea movido de lugar, y las referencias de los programas a estas librerías sean inválidas.

13. Considere un sistema de paginación en el que la tabla de páginas esté almacenada en memoria.

a. Si una referencia a memoria tarda en realizarse 50 nanosegundos, ¿cuánto tiempo tardará una referencia a memoria paginada?

- 50 ns para traer la tabla de páginas.
- 50 ns para referenciar la referenciar la página.
- Entonces se tardará un total de 100ns una referencia a memoria paginada.

b. Si añadimos buffers TLB y el 75 % de todas las referencias a las tablas de páginas se encuentran en los buffers TLB, ¿cual es el tiempo efectivo que tarda una referencia a memoria? (Suponga que la localización a una entrada de la tabla de páginas contenida en los buffers TLB se hace en tiempo cero, si la entrada ya se encuentra allí).

Tiempo efectivo: $\text{Hit Rate} * 50\text{ns ref mem} + \text{Miss Rate} * 100\text{ns ref mem}$

Tiempo efectivo: $75\% * 50\text{ns} + 25\% * 100\text{ns}$

Tiempo efectivo: $37,5\text{ns} + 25\text{ns} = 62,5\text{ns}$

14. ¿En qué caso utilizaría página invertida? ¿Por qué?

Silberschatz 9.4.3

Tannenbaum 3.3.4

Cuando el espacio de direccionamiento virtual es mucho más grande que la memoria física, lo que ocasiona que se gaste mucho espacio en tablas de páginas para cada proceso. El principal problema que se tiene es que se gasta mucho tiempo buscando la página en la tabla de frames, esto se puede minimizar haciendo uso correcto del TLB. Además, hay que tener cuidado con que no se asignen dos páginas de dos distintos espacios direccionables a un mismo frame.

Usualmente, cada proceso tiene su tabla de páginas asociada. La tabla de páginas tiene una entrada por cada página que el proceso está usando. Una de las desventajas de este método es que cada tabla de páginas puede consistir de millones de entradas. Estas tablas pueden consumir grandes cantidades de memoria física solamente para hacer seguimiento de cómo otra memoria física es usada.

Para resolver este problema y este caso, podemos usar entonces una tabla de páginas invertida. Una tabla de páginas invertida tiene una entrada por cada página real (o frame) de memoria. Cada entrada consiste de direcciones virtuales de la página almacenada en esa locación de memoria real. Entonces, solamente hay una tabla de páginas en el sistema, y tiene solamente una entrada por cada página en memoria física. La tabla de páginas invertida generalmente requiere que un identificador de espacio de direcciones sea almacenado en cada entrada de la tabla de páginas, ya que usualmente contiene varios espacios de direcciones diferentes mapeando la memoria física. Almacenando el identificador de espacio de memoria asegura que una página lógica para un proceso en particular sea mapeado al frame de página físico correspondiente.

Aunque este esquema reduce la cantidad de memoria requerida para almacenar cada tabla de páginas, incrementa la cantidad de tiempo necesario para buscar la tabla cuando una referencia a una página ocurre. Dado que la tabla de páginas invertidas se ordena por dirección física, pero las búsquedas se realizan en direcciones virtuales, es posible que sea necesario realizar una búsqueda en toda la tabla antes de encontrar una coincidencia. Esta búsqueda llevaría demasiado tiempo. Se usa la función hash para esto.

Y algo importante que nos dice el libro, es que no se usaría cuando queremos tener el caso de las páginas compartidas, debido a que solo hay una entrada de página virtual por cada página física, una página física no puede tener dos (o más) direcciones virtuales compartidas. Por lo tanto, con tablas de páginas invertidas, sólo puede ocurrir una asignación de una dirección virtual a la dirección física compartida en un momento dado.

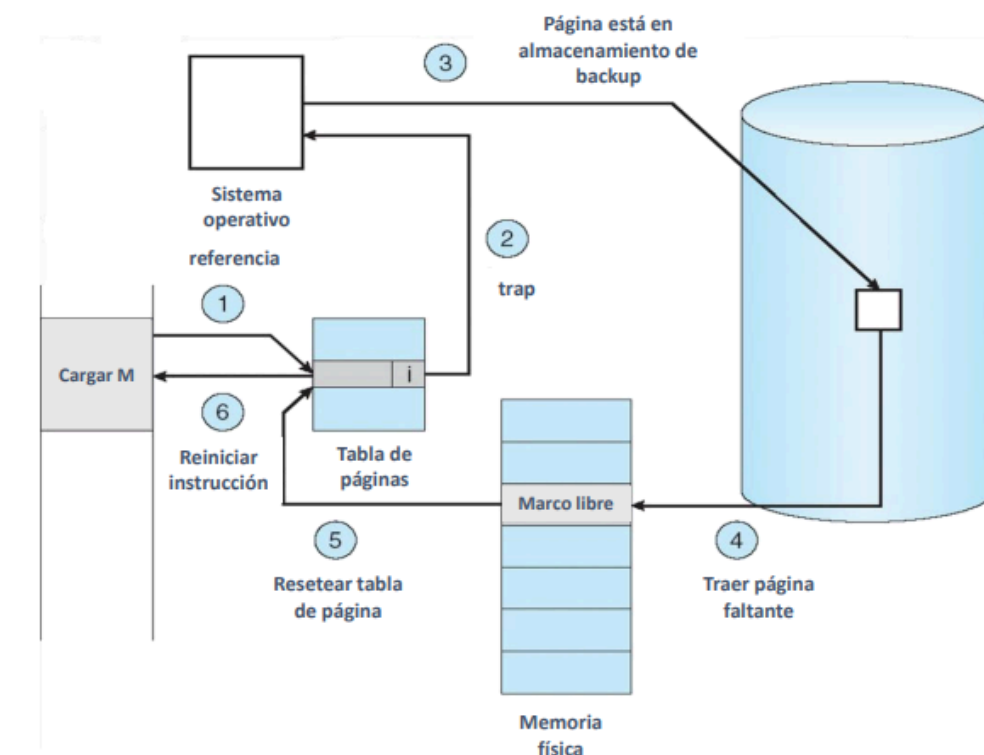
Una referencia por parte de otro proceso que comparta la memoria generará un error de página y reemplazará la asignación con una dirección virtual diferente.

15. Describe las acciones que realiza un sistema operativo cuando ocurre un fallo de página. ¿Cuándo y cómo interviene el sistema operativo?

Silberschatz 10.2.1

Las acciones que realiza el sistemas operativo son, primero al acceder a un entrada invalida a la tabla de páginas causa un page-fault, lo que acciona un trap al sistema operativo y la forma que tiene de manejar este trap es directa:

1. Chequea una tabla interna (usualmente en el PCB) para este proceso para determinar si la referencia fue un acceso a memoria válida o inválida
2. Si la referencia fue inválida, terminamos el proceso. Si fue válida pero no tenemos aún la pagina, traemos la página a memoria.
3. Buscamos un frame libre.
4. Programamos una operación en memoria secundaria para leer la página deseada en el frame recién designado.
5. Cuando la lectura del almacenamiento es completada, modificamos la tabla interna del proceso y la tabla de página donde ahora la página está en memoria.
6. Reiniciamos la instrucción que fue interrumpida por el trap. El proceso ahora puede acceder a la página como si siempre la hubiera tenido en memoria. Es importante que los sistemas operativos tengan esta habilidad de restaurar una instrucción después de un page fault en paginación por demanda. Tienen que saber resguardar el proceso a exactamente como estaban al momento de la falta en términos de lugar y estado, a menos que ya esté en memoria y sea accesible.



16. Consideremos un sistema de intercambio en el que la memoria consta de los siguientes tamaños de espacios en orden de memoria: 10K, 4K, 20K, 18K, 7K, 9K, 12K y 15K. ¿Cuál es el espacio que se elige si se solicitan en forma sucesiva segmentos de 12K, 10K y 9K?

Mejor ajuste: 12K, 10K y 9K.

Primer ajuste: El tercero (20K) , el primero (10K), el cuarto (18K).

Siguiente ajuste: 20K, 18K, 9K.

Peor ajuste: 20K, 18K, 15K.

4. First fit takes 20 MB, 10 MB, 18 MB. Best fit takes 12 MB, 10 MB, and 9 MB. Worst fit takes 20 MB, 18 MB, and 15 MB. Next fit takes 20 MB, 18 MB, and 9 MB.

17. Suponga que tiene un tamaño de memoria de 1024 KB y la memoria se asigna utilizando el algoritmo de Buddy (del compañero).

a. Muestre la asignación de memoria para las siguientes solicitudes:

- i. Solicita 240 KBytes
- ii. Solicita 120 KBytes
- iii. Solicita 60 KBytes
- iv. Solicita 130 KBytes

1024 KB					
512KB			512KB		
240KB	256KB		512KB		
240KB	120KB	128KB	512KB		
240KB	120KB	60	64	512KB	
240KB	120KB	60	64	130KB (126KB sin usar)	256KB

b. Muestre la liberación de memoria para las siguientes solicitudes:

- i. Libera 240 KBytes
- ii. Libera 60 KBytes
- iii. Libera 120 KBytes

Partiendo del estado inicial:

240KB	120KB	60	64	130KB (126KB sin usar)	256KB
-------	-------	----	----	------------------------	-------

256KB	120KB	60	64	130KB (126KB sin usar)	256KB
-------	-------	----	----	------------------------	-------

256KB	120KB	128KB		130KB (126KB sin usar)	256KB
-------	-------	-------	--	------------------------	-------

512KB				130KB (126KB sin usar)	256KB
-------	--	--	--	------------------------	-------

18. Si se utiliza el reemplazo FIFO con cuatro marcos y ocho páginas, ¿Cuántos fallos ocurrirán con la cadena de referencia 0,1, 7, 2, 3, 2, 7, 1, 0, 3 si los cuatro marcos están vacíos al principio? Repita el problema con LRU.

6 fallos con FIFO (First-In-First-Out).

Secuencia	0	1	7	2	3	2	7	1	0	3
	0	0 1	0 1 7	0 1 7 2	3 1 7 2	3 1 7 2	3 1 7 2	3 1 7 2	3 0 7 2	3 0 7 2
Fallos	1	2	3	4	5	5	5	5	6	6

7 fallos con LRU (Least-Recently-Used).

Secuencia	0	1	7	2	3	2	7	1	0	3
	0	0 1	0 1 7	0 1 7 2	3 1 7 2	3 1 7 2	3 1 7 2	3 1 7 2	0 1 7 2	0 1 7 3
Fallos	1	2	3	4	5	5	5	5	6	7

19. Si una instrucción tarda 1 useg. y un fallo de página tarda n useg. adicionales, de una fórmula para el tiempo promedio de las instrucciones si los errores aparecen cada k instrucciones.

$$1 \text{ useg} + \frac{n}{k} \text{ useg}$$

13. A page fault every k instructions adds an extra overhead of n/k μsec to the average, so the average instruction takes $1 + n/k$ nsec.

20. ¿Es necesario proteger la memoria en un sistema de memoria virtual? ¿Por qué?

Sí que es necesario proteger la memoria en un sistema de mv, para ello contamos con los bits adicionales en la tabla de páginas, ya sean los de validez o algunos permisos de lectura, escritura, etc. Si un proceso quiere acceder a una página pero esta está marcada como inválida, entonces el proceso se debe abortar la acción.

Esto se debe a que supongamos que la información que hay allí es privada y cómo ya acabó el proceso que la usaba se volvió inválida. Entonces el nuevo proceso no tiene porqué poder acceder a esa información, por más que se encuentre en el mismo número de página ya que se podrían estar accediendo a datos que no se deberían y no se sabe que se puede hacer con eso. Es por eso que primero que todo se tiene que revisar si la información ubicada allí es válida y luego revisar los otros bits. **RE DUDOSA LA RESPUESTA XD**

Encontré esto:

La segmentación se presta por sí misma a la implementación de políticas de compartir y protección. Ya que cada entrada de tabla de segmentos incluye una longitud así como su dirección base, un programa no puede acceder de manera inadvertida a una locación de memoria principal más allá de los límites del segmento.

Para el paginado, la estructura de los programas y datos de la página no son visibles para el programador, haciendo la especificación de protección y compartir más incómoda.

Pueden ser provistos mecanismos más sofisticados. Un esquema común es usar una estructura de anillo de protección. En este esquema, los anillos con numeración baja (o 13 / 16 internos) gozan de mayores privilegios que los con numeración alta (o externos).

Típicamente, el anillo 0 es reservado para las funciones del kernel del SO, con aplicaciones en los niveles superiores. Algunas utilidades o servicios del SO pueden ocupar un anillo intermedio. Los principios básicos para el sistema de anillos son:

- Un programa puede acceder solamente a información que resida en el mismo anillo o en uno con menos privilegios
- Un programa puede llamar servicios que residan en el mismo anillo o en uno con mayores privilegios

21. Un sistema de 32 bits utiliza una tabla de páginas de dos niveles. Las direcciones virtuales se dividen en un campo de 9 bits para la tabla de nivel superior y un campo de 11 bits para la tabla de nivel secundario, además de un ajuste. ¿Cuál es el tamaño de las páginas y cuál es el número en el espacio de direcciones virtuales?

9 bits superior	11 bits inferior	12 bits
Nro de Página		Offset

$32 - 9 - 11 = 12 \Rightarrow 2^{12}$ bytes de tamaño de página (con referencia al byte) = 4KB
 $11 + 9 = 20$ bits de paginado $\Rightarrow 2^{20}$ cantidad de páginas = 1,048,576 páginas

Espacio de direcciones virtuales $\Rightarrow$ Sistema de 32 bits

22. Una máquina tiene direcciones virtuales de 48 bits y direcciones físicas de 32 bits. Las páginas son de 8K. ¿Cuántos datos se necesitan para una tabla de páginas convencional? ¿Para una tabla invertida? Considere que tiene los siguientes bits de control: válido/inválido, referenciado, modificado, lectura, escritura.

Se tienen páginas de 8 KB $\rightarrow$ 8192 Bytes (2^{13}) $\rightarrow$ 13 bits de offset.
Entonces, $32 - 13 = 19$ bits de número de frame = 2^{19} frames.
 $48 - 13 = 35$ bits de número de página = 2^{35} páginas.

Para una tabla de páginas convencional, se necesitan:

- Bit de válido/inválido
- 19 bits de nro de frame
- Bit de referenciado
- Bit de modificado
- Bit de lectura
- Bit de escritura

Tendrá un total de 2^{35} entradas.

Para una tabla de páginas invertida, se necesitan:

- Bit de válido/inválido
- 35 bits de nro de página
- Bit de referenciado
- Bit de modificado
- Bit de lectura
- Bit de escritura

Tendrá un total de 2^{19} entradas.

En base a las respuestas del libro considera esto de las entradas en tablas de páginas invertidas y convencionales.

9.8 The BTV operating system has a 21-bit virtual address, yet on certain embedded devices, it has only a 16-bit physical address. It also has a 2-KB page size. How many entries are there in each of the following?

Conventional, single-level page table will have $2^{10} = 1024$ entries.
Inverted page table will have $2^5 = 32$ entries. The maximum amount of physical memory is $2^{16} = 65536$ (or 64 KB.)

23. Se tiene la siguiente situación:

- Acceso al TLB 20 ns.
- Acierto al TLB 80%.
- Tiempo de acceso a memoria 75 ns.

- Tiempo de swap de una página 500,000 ns.
- El 50% de las páginas están sucias. (P_{uaj})
- El sistema operativo utiliza una tabla de páginas de un sólo nivel.

Fórmulas

TEA TLB = Acceso TLB + (Hit Rate * Acceso a Memoria) + (Miss Rate * 2 * Acceso a Memoria)

TEA = (1 – p) x acceso a memoria + p x (sobrecarga de falta de página + salida de la página + entrada de la página + sobrecarga de reinicio)

a. ¿Cuál es el tiempo de acceso efectivo, si se asume que el porcentaje de fallo de página es 0%?. Asuma que los costos de actualización son despreciables. (p = porcentaje fallo de página)

Hay dirty bit, pero como el costo de actualización es despreciable, no importa (?)

Hay que hacerlo de alguna manera para tener en cuenta que el acceso al tlb es paralelo a la tabla de páginas?

$T_{\text{acceso efectivo Piuma}} = 0,8 \cdot (20\text{ns} + 75\text{ns}) + 0,2 \cdot 75\text{ns} \cdot 2$
(es esto nomas, ya que la probabilidad de page fault es 0)

b. ¿Cuál es el tiempo de acceso efectivo, si se asume que el porcentaje de fallo de página es 10 %?. Asuma que los costos de actualización son despreciables.

$T_{\text{acceso efectivo}} = 0,8 \cdot (20\text{ns} + 75\text{ns}) + 0,2 \cdot 75\text{ns} \cdot 2 \cdot 0,9 + 0,2 \cdot 0,1 \cdot (75\text{ns} \cdot 2 + 500,000\text{ns})$

24. Suponga un esquema de memoria paginada con Memoria Virtual. Se están ejecutando tres programas A, B y C con longitudes de 2K, 1.5K, y 3K caracteres respectivamente. La longitud de la página es de 0.5K caracteres.

a. ¿Cuál es la cantidad máxima de páginas que puede tener un programa si las instrucciones tienen direcciones de 16 bits?

0,5 KB → 512 Bytes → 2^9 Bytes → 9 bits de offset → 16 - 9 = 7 bits de nro de pág → 2^7 páginas → 128 páginas

b. Determine el contenido de las tablas para los programas A, B y C. Los bloques 0 a 8 de la memoria contienen las páginas A-0, B-0, C-5, A-1, A-2, C-3, C-1, B-2 y C-4.

Bloques de memoria

0	A-0
---	-----

1	B-0
2	C-5
3	A-1
4	A-2
5	C-3
6	C-1
7	B-2
8	C-4
	...
Y	X

Tabla de páginas proceso A

0	Frame 0
1	Frame 3
2	Frame 4
3	Page Fault

Tabla de páginas proceso B

0	Frame 1
1	Page Fault
2	Frame 7

Tabla de páginas proceso C

0	Page Fault
1	Frame 6
2	Page Fault
3	Frame 5
4	Frame 8
5	Frame 2

c. En la dirección X'0280' del programa A hay una instrucción de bifurcación incondicional. Utilizando las tablas definidas en anterior determine la dirección de la memoria en la cual está la instrucción. Indique ahora cómo actúa el sistema, utilizando las tablas si es necesario, si en la instrucción se bifurca a la X'029A', a la X'00BA' y a la X'0708'. Cada uno de los caracteres que aparece en la dirección corresponde a 4 bits, y está definido en hexadecimal.

Página 1 de A, offset 128, dirección de memoria 3 y offset 128

Bifurcaciones:

- 029A Pagina 1 (frame 3), offset 154
- 00BA Pagina 0 (frame 0), offset 186
- 0708 Pagina 3 (page fault, la trae de disco), offset 264

25. Se desea implementar una gestión de memoria virtual mediante paginación en un equipo. Las direcciones lógicas son de 32 bits. La unidad mínima direccionable es un byte. La cantidad de memoria RAM instalada en el equipo puede ser variable. La cantidad mínima de memoria es de 4 MB y se puede incrementar en potencias de 2 (es decir 4MB, 8MB, 16MB, 32MB, etc..., hasta un máximo). El tamaño de la página es de 4 KB y el del descriptor de página de 3 bytes (entrada en la tabla). Se desea poder controlar si se ha accedido a una página, si está presente, si se ha modificado, así como también en poder protegerla de lectura, escritura y ejecución. Con estos datos en principio, supuesto que se dispone de suficiente memoria secundaria para dar soporte a la memoria virtual y para un espacio de direccionamiento único se pide:

a. Especifique los formatos de dirección lógica (virtual) y de descriptor de página.

Tamaño página 4KB → 4096 Bytes → 2^{12}

Dirección virtual: 20 bits nro pag y 12 bits offset

20 bits	12 bits
Nro de pág	Offset

Descriptor de página (3 Bytes = 24 bits): 1 bit acceso, 1 bit válido, 1 bit dirty, 1 bit lectura, 1 bit escritura, 1 bit modificado y 18 bits nro frame (el resto).

18 bits	1 bit	1 bit	1 bit	1 bit	1 bit	1 bit
Nro Frame	Acceso	Válido	Dirty	Lectura	Escritura	Modific.

b. *Escriba el tamaño de la tabla de páginas (en bytes):*

Tamaño de la tabla de páginas = tamaño de la entrada * cantidad de páginas

Tamaño de la tabla de páginas = 3 Bytes * 2^{20} paginas = 3145728 Bytes $\approx$ 3 MB

c. *Escriba la cantidad máxima de memoria real que puede manejar este Sistema (en bytes).*

La cantidad máxima está limitada por el número de bits de frame en la tabla de páginas (en este caso), por lo que sería necesario la siguiente memoria real para manejar el sistema:

Total bits dirección física = 18 bits nro frame + 12 bits offset = 30 bits

Tamaño total memoria física = 2^{30} Bytes = 1.073.741.824 Bytes = 1GB (Literal es el tamaño clavado de lo que equivale un giga en bytes)

Normalmente, está limitado por el espacio direccionable (en este caso 32 bits), en cuyo caso sería 4 GB. Analizando para diferentes valores de memoria RAM No entendi lo de rojo

- 4MB $\rightarrow 2^2 * 2^{10} * 2^{10} = 2^{22}$
- 8MB $\rightarrow 2^3 * 2^{10} * 2^{10} = 2^{23}$
- 16MB $\rightarrow 2^4 * 2^{10} * 2^{10} = 2^{24}$
- 32MB $\rightarrow 2^5 * 2^{10} * 2^{10} = 2^{25}$
- ...
- 1024MB $\rightarrow 2^2 * 2^{10} * 2^{10} = 2^{30}$ cantidad máxima de memorial real. 1GB

Para dar soporte a la memoria virtual se dispone de un dispositivo de almacenamiento secundario de 8GB, pero que solo tiene disponibles 2GB para memoria virtual. Estos 2GB están asignados a un fichero que se usará para guardar las páginas. Los bloques de datos de este fichero no tienen por qué ser contiguos.

Cuando una página no se encuentra en memoria real, el campo "Número de marco" del descriptor de página guarda el número de bloque del dispositivo de almacenamiento

secundario donde reside la página. La numeración de bloques del disco comienza en '1', así que se reserva el número '0' para indicar que la página no tiene ningún bloque asignado. Con esta nueva información se pide:

a. *Escriba la cantidad de memoria virtual máxima de la que se podrá disponer (en bytes).*

Como ahora se tiene que si no está en memoria real, está en virtual, entonces contamos con 2GB disponibles para la memoria virtual, lo que da:

$$2\text{GB} = 2^{31} \text{ Bytes}$$

(Podría considerarse menos uno por el bit que nos indica si la página tiene algún bloque asignado).

b. *Escriba el tamaño que debería tener el bloque de almacenamiento secundario para dar soporte a la misma (en bytes).*

Si la página no está en memoria real el número de frame se corresponde con el número del bloque del almacenamiento secundario, por lo tanto se tienen un total de 2^{18} bloques sacado de la cantidad de bits del número de frame.

Si se almacena un frame por bloque, y cada bloque tiene de tamaño 4KB entonces se tienen: $2^{18} * 4\text{KB} = 1\text{GB}$ de datos ocupados, por lo que sobraría 1GB de almacenamiento secundario (1GB almacenamiento libre + 1GB destinado a dar soporte).

26. Se desea implementar un sistema operativo con multiprogramación y memoria virtual paginada con las características siguientes:

- Un único espacio de memoria virtual de 4GB a compartir por todos los procesos que se creen.
- El bus de datos es de 32 bits.
- MMU con caché interna (TLB) de 64 entradas que incluyen los bits R (Referenciada) y M (Modificada).
- Una memoria principal de 512MB.
- Páginas de 8 KB.
- Tabla de Páginas Invertidas cargada completamente en memoria principal.
- Mapa de Bits para llevar el control de la memoria virtual libre y ocupada.
- Algoritmo LRU (aproximado) como política de sustitución de páginas.

Responder, a las siguientes preguntas:

a. Indicar el formato (campos con su significado y tamaño en bits) de un descriptor de la Tabla de Páginas Invertida (supuesto que cada descriptor debe ocupar un número entero de bytes) y calcular el tamaño en bytes de dicha tabla.

Bits nro pag: 32 bits - 13 bits frame = 19 bits nro pag

2^{16} cantidad frames * (19 bits nro pag + 1 bit ref + 1 bit mod) =

$$2^{16} * 3 \text{ bytes} = 192 \text{ KB}$$

b. Indicar el tamaño en bytes que ocupa el Mapa de Bits que lleva el control de la memoria virtual libre y ocupada.

Como hay 2 a la 19 páginas, el mapa de bits ocupa 2 a la 19 bits entonces 2 a la 16 B => 64 KB

c. Indicar el tipo de estructura de datos que se crea más aconsejable para llevar el control de la memoria principal libre y ocupada (marcos de página) de forma que se agilice la búsqueda de un marco libre cuando se produzca falta de página. Calcular el tamaño en bytes de esta estructura supuesto que una variable o campo de una estructura debe ocupar un número entero de bytes.

Hay tres opciones:

- Mapa de bits
- Lista enlazada
- Counting (tener un bloque que lleve la cuenta de a continuación cuántos bloques libres hay)
- Grouping (tener un bloque que diga de los siguientes n bloques, cuántos libros hay, y el bloque n+1 reinicia la cuenta para los siguientes).

d. Cuando se utiliza únicamente una Tabla de Páginas clásica (no invertida y no multinivel) que está cargada en memoria principal y una MMU que cuenta con RBTP (Registro Base de la Tabla de Páginas) así como su correspondiente TLB, el número máximo de accesos a memoria principal cuando se intenta acceder a una dirección virtual que pertenece a una página presente en memoria principal es de tres. Indicar cuando se produce esta situación y a que se deben los tres accesos.

Como la tabla de paginas normal terminaria siendo muy grande, podria pasar lo siguiente:

Como era??

1er acceso: en el tlb todas las entradas tienen el dirtybit =1, tiene que guardar ese bit en la tabla de paginas antes de reemplazar la entrada

2do acceso: a Tabla de Páginas usando RBTP

3er acceso: acceso al frame donde estaba la página.

e. Cuando se utiliza una Tabla de Páginas Invertida, como en nuestro caso, la situación descrita en el punto anterior (4), en general, no puede resolverse con tan solo tres accesos a

memoria principal. Indicar por qué y cuál podría ser (con los datos del enunciado) el número máximo de accesos a memoria principal.

Esto no se puede hacer porque hay que recorrer completamente toda la tabla hasta encontrar el frame en el cual está la página. En el peor caso, en donde la página está en el último frame, pasa lo siguiente:

$192 \text{ KB} / 8 \text{ KB tam frame} = 24 \rightarrow$ la tabla de frames está en 24 páginas distintas $\rightarrow$ 25 accesos a memoria (a las 24 partes de la tabla de páginas y finalmente a memoria a buscar la página).

27. Es una máquina de 64 bits que utiliza una MMU de 48 bits y administración de memoria con segmentación paginada con varios niveles. Las direcciones se dividen de la siguiente forma:

segmento	1er. Nivel Tabla Página	2do. Nivel Tabla Página	Desplazamiento
6 bits	14 bits	14 bits	14 bits

a. *Indique el tamaño de página.*

$2^{14} \text{ Bytes} = 16384 \text{ Bytes} = 16 \text{ KB}$

b. *Indique la cantidad de páginas totales que tiene el espacio virtual.*

$14+14 = 28 \rightarrow 2^{28}$ páginas totales por segmento (ya que cada segmento tiene sus propias tablas de páginas). Como se pueden tener 2^6 segmentos, entonces se tienen 2^{34} páginas totales disponibles en el espacio virtual (en total).

c. *Muestre el descriptor de entrada para la tabla del segmento y la tabla del 2do. nivel. Estime el tamaño de cada tabla.*

Entrada tabla segmento:

Bits de Control	Longitud	Base segmento
-----------------	----------	---------------

Suponiendo que tiene 3 bits de permiso, uno de válido, uno de modificación y otro de visitado, entonces:

Tamaño de la tabla = $(6 \text{ bits} + 14 \text{ bits}) * 2^6 = 3 \text{ Bytes} * 2^6 = 192 \text{ Bytes}$

Entrada tabla del 2do nivel:

Presente	Modificado	Bits de Control	Número de frame
----------	------------	-----------------	-----------------

Para el número de frame, supongamos que se tiene todo el espacio direccionable (2^{64}) disponible de manera física. Entonces, serían $64 - 14 \text{ offset} = 2^{50}$ frames totales (y 50 bits para direccionarlos).

Tamaño de la tabla = $2^{14} * (6 \text{ bits adicionales} + 50 \text{ bits frame}) = 2^{14} * 7 \text{ Bytes} = 114688 \text{ Bytes} = 112 \text{ KB}$

d. *Indique el número total de páginas que puede tener el segmento.*

$2^{14} + 2^{14} = 2^{14+14} = 2^{28}$ páginas totales puede tener.

28. El sistema VAX/VMS utiliza un algoritmo de sustitución FIFO para las páginas residentes y un conjunto compartido de marcos libres compuesto por páginas recientemente utilizadas. Suponga que el conjunto compartido de marcos libres se gestiona utilizando la política de sustitución menos recientemente utilizadas.

Páginas residentes: páginas que están en la memoria principal y pueden ser accedidas directamente por el procesador.

Grupo de marcos libres: son marcos de memoria en la RAM que no están asignados a ninguna página residente en ese momento, es decir, están vacíos y disponibles para ser usados cuando sea necesario. Estos marcos libres pueden ser utilizados para cargar nuevas páginas de memoria, pero no contienen páginas activas o en uso.

a. *Si se produce un fallo de página y la página no se encuentra en el conjunto compartido de marcos libres, ¿cómo puede generarse espacio libre para la nueva página solicitada?*

Se puede generar espacio libre expulsando algunas de las páginas del grupo de marco libre y llevar estas hacia el disco, creando así espacio para que una de las páginas residentes se mueva al grupo de marcos libres. A continuación, la página a la que se accede se la mueve al conjunto residente que previamente hicimos lugar.

b. *Si se produce un fallo de página y la página se encuentra en el conjunto compartido de marcos libres, ¿cómo se activa la página residente y cómo se gestiona el conjunto compartido de marcos libres para hacer sitio para la página solicitada?*

Si ocurre esta situación, entonces lo que se hace es traer esta página que se encuentra en el conjunto de marcos libres hacia el conjunto de páginas residentes, mientras que a su vez una de las páginas residentes es movida hacia el grupo de páginas libres.

c. *Si se configura el número de páginas residentes con el valor uno, ¿cuál modelo de algoritmo sería?*

Si se hiciera esto y el número de páginas residentes se establecieran en uno, entonces el sistema se degenera en el algoritmo de reemplazo de página usado en el grupo de marcos libres, es decir en LRU.

d. Si se configura el número de páginas del conjunto compartido de marcos libres en cero, ¿cuál modelo de algoritmo será?

En este caso ocurre la inversa de lo anterior, ya que el sistema degenera en un algoritmo de reemplazo de páginas FIFO cuando el conjunto compartido de marcos libres es 0.